

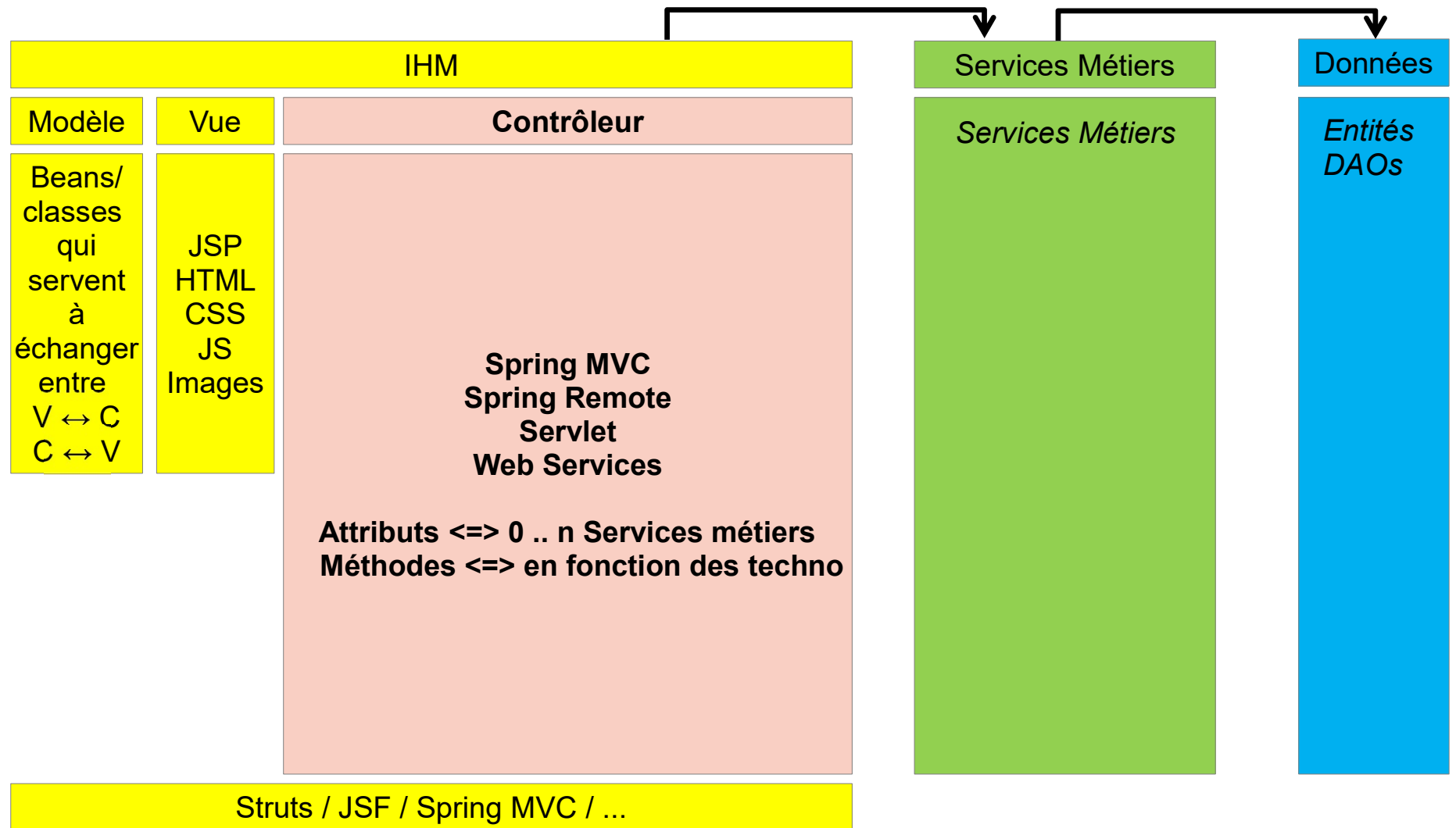


Spring MVC

En mode Web Service REST

1

Présentation



Spring MVC

Web Service REST

3

- ✓ Présentation
- ✓ Rappels
 - ✓ Mapping XML
 - ✓ Mapping JSON
- ✓ Norme JAX
- ✓ Framework JSON
- ✓ Rappels :
 - ✓ HTTP
 - ✓ Commandes Get, Put, ...
 - ✓ Un web service REST
 - ✓ Problématique d'authentification
 - ✓ Problématique de documentation
- ✓ Annotations Spring MVC
- ✓ Gestion des erreurs
- ✓ HAL et HATEOAS
- ✓ Spring et Ajax

Mapping

➤ **Définition** : prendre un objet pour le transformer en flux et inversement, prendre un flux pour le transformer en objet

❑ **Mapping XML** : Transformer un objet en flux XML et/ou un flux XML en objet

❑ **Mapping JSON** : Transformer un objet en flux JSON et/ou un flux JSON en objet

Rappels XML

➤ XML : eXtensible Markup Language

- ❑ Langage de balisage
- ❑ Extensible

- Un langage initialement destiné à publier de l'information et à échanger des données sur le Web
- Une recommandation du W3C (World Wide Web Consortium)
- Né dans les années 1996, adopté officiellement en octobre 2000

Rappels XML

➤ Absence de balises prédéfinies

- ❑ C'est à vous de choisir les balises et les attributs

➤ Un méta-langage permettant de prédéfinir

- ❑ des jeux de balises
- ❑ des règles d'utilisation pour les jeux de balises

➤ Rendre les données exploitables par diverses applications

- ❑ Présentation à des utilisateurs
- ❑ Manipulation et traitements automatisés
- ❑ Echanges entre applications

Exemple XML

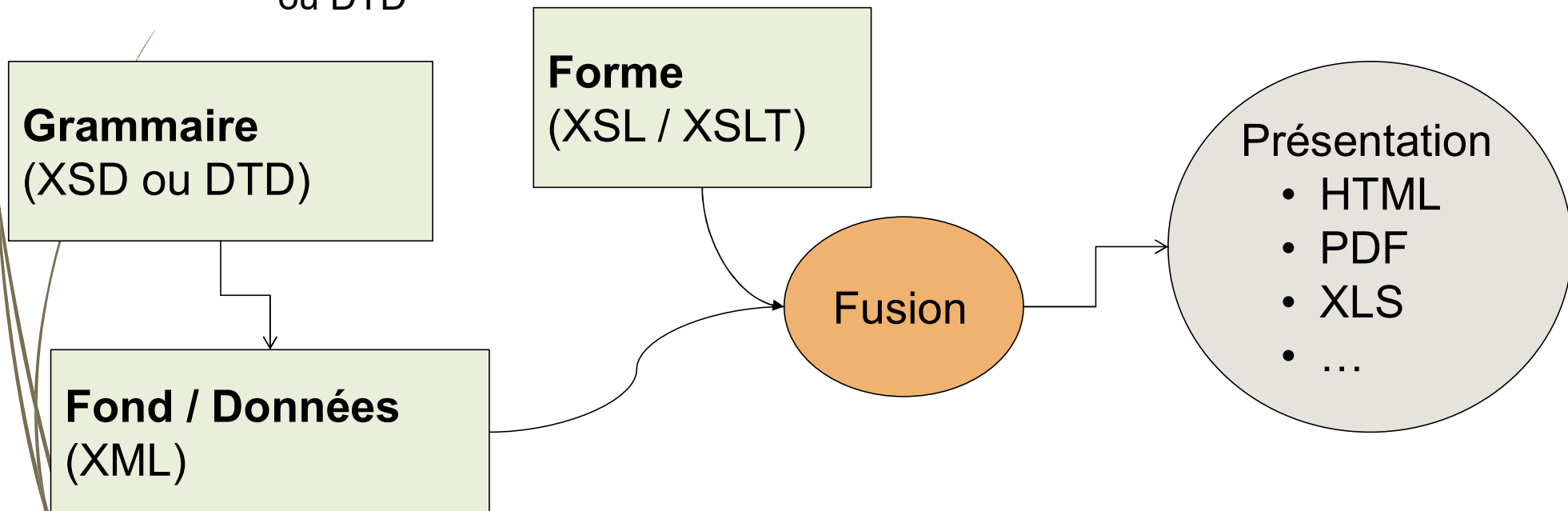
```
<Catalogue saison="2001-2002">
  <formation filiere="internet">
    <animateur nom1="Barone"/>
    <titre>XML - les fondamentaux</titre>
    <duree>1 jour</duree>
    <type>Concepts</type>
    <prix>500e HT</prix>
  </formation>
  <formation>
    ...
  </formation>
</Catalogue>
```

XML

8

➤ XML sépare le fond de la forme

- Le fond définit le contenu, les données, transmises (XML).
- La forme est fournie par un langage de style (XSL,CSS) ou du Java, .NET, JS,
- La grammaire définit la structure des données, elle est définie dans un fichier XSD ou DTD



Structure du document XML

- Un document XML est structuré au moyen de balises
 - Délimitées par les symboles '<' et '>'
 - ❑ Exemple : <balise>
 - Refermées par la même balise commençant par '</'
 - ❑ Exemple : </balise>
 - Son contenu est situé entre ces deux délimiteurs
 - Obéissant à une syntaxe stricte
- Un document XML se compose
 - D'un prologue, optionnel, mais recommandé. On y trouve divers types de déclarations (doctype, version, encoding ...)
 - D'un arbre d'éléments (les balises)
 - De commentaires, d'instructions de traitement, facultatifs et pouvant apparaître dans les deux parties précédentes

Structure du document XML

➤ Les documents **bien-formés**

- Obéissent aux règles syntaxiques de XML. Ils sont exploitables par un parser.
- Sont conformes, mais pas forcément élégants ou pratiques !

➤ Les documents **valides**

- Sont toujours bien formés
- Respectent le modèle de document explicité dans une DTD ou XSD
- Peuvent être utilisés sous forme de simples documents bien-formés (navigateur, ...)

➤ Il existe des analyseurs syntaxiques validant et non validant

Le document XML

➤ Références

➤ A des caractères

- ❑ Ce sont des références directes à des valeurs dans des jeux de caractères
- ❑ Elles sont considérées comme du character data
- ❑ Elles sont utiles si vous ne pouvez pas taper le caractère souhaité

➤ A des entités prédéfinies

- ❑ <, >, &, ', ";
- ❑ resp. <, >, &, ', "

```
<Dpt>Etudes &#38; Conseils Technologiques</Dpt>  
<Dpt>Etudes &x003c; Conseils Technologiques</Dpt>  
<Dpt>Etudes &amp; Conseils Technologiques</Dpt>
```

Le document XML

➤ Éléments ou balise

- Ce sont des containers (<tag></tag>)
- Ils peuvent s'imbriquer pour former un arbre (père / enfants)
- Ils peuvent être vides (<tag />)

```
<Catalogue saison="2001-2002">
  <formation filiere="internet">
    <animateur nom1="Barone"/>
    <titre>XML - les fondamentaux</titre>
    <duree>1 jour</duree>
    <type>Concepts</type>
    <prix>500e HT</prix>
  </formation>
</Catalogue>
```

Le document XML

➤ Attributs

- Ce sont les caractéristiques d'un élément
- Ils sont de la forme : nom = valeur
- Leur utilisation doit être "élégante" (concise et adaptée au traitement)

```
<Catalogue saison="2001-2002">  
  <formation filiere="internet">  
    <animateur nom1="Barone"/>  
    <titre>XML - les fondamentaux</titre>  
    <duree>1 jour</duree>  
    <type>Concepts</type>  
    <prix>500e HT</prix>  
  </formation>  
</Catalogue>
```

Le document XML

➤ Commentaires

➤ Ce sont des balises silencieuses `<!-- -->`

```
<Catalogue saison="2001-2002">
  <!-- Mois de début et mois de fin ? -->
  <formation filiere="internet">
    <animateur nom1="Barone"/>
    <titre>XML - les fondamentaux</titre>
    <duree>1 jour</duree>
    <type>Concepts</type>
    <prix>500e HT</prix>
  </formation>
  <formation>
    ...
  </formation>
</Catalogue>
```

XML-Schema (XSD)

➤ Catégorie

- Définition de modèle de document

➤ Statut

- Recommandation (2 mai 2001)

➤ Objectif

- Dépasser les limitations des DTD en exprimant la structure et les contraintes sur les types de données du document sous forme XML

XML-Schema (XSD)

- XML-Schema est un langage XML qui :
 - Se substitue aux DTD (non XML) pour définir la structure du document
 - Étend les DTD concernant les types et formats de données et gère les espaces de noms
 - Introduit la notion d'héritage entre éléments XML.
- Plusieurs tentatives précédentes
 - DCD
 - XML-Data

XML-Schema (XSD)

➤ Principes

- Une description de type de document XML écrit en XML-Schema est un document XML dont
 - ❑ L'élément document est <schema>
 - ❑ Indique sa propre URI (gère les versions)
 - ❑ Définit les types d'éléments, d'attributs, de données
- Les types de données XML-Schema peuvent être
 - ❑ Des types prédéfinis
 - Types XML 1.0 (DTD) : ID, ENTITY, NOTATION, ...
 - string, binary, boolean, number, integer, decimal, real, dateTime, date, time, timePeriod, uri
 - ❑ Des types de données "utilisateur"

XML-Schema (XSD)

18

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:tns="http://tempuri.org/PurchaseOrderSchema.xsd"
  targetNamespace="http://tempuri.org/PurchaseOrderSchema.xsd"
  elementFormDefault="qualified">

  <xsd:element name="PurchaseOrder" type="tns:PurchaseOrderType"/>
  <xsd:complexType name="PurchaseOrderType">
    <xsd:sequence>
      <xsd:element name="ShipTo" type="tns:USAddress" maxOccurs="2"/>
      <xsd:element name="BillTo" type="tns:USAddress"/>
    </xsd:sequence>
    <xsd:attribute name="OrderDate" type="xsd:date"/>
  </xsd:complexType>

  <xsd:complexType name="USAddress">
    <xsd:sequence>
      <xsd:element name="name" type="xsd:string"/>
      <xsd:element name="street" type="xsd:string"/>
      <xsd:element name="city" type="xsd:string"/>
      <xsd:element name="state" type="xsd:string"/>
      <xsd:element name="zip" type="xsd:integer"/>
    </xsd:sequence>
    <xsd:attribute name="country" type="xsd:NMTOKEN" fixed="US"/>
  </xsd:complexType>
</xsd:schema>
```

JAX

19

- La norme JAX : **JA**va **X**ml
- Regroupe tout ce qui concerne
 - La gestion du XML (mapping, parsing)
 - La gestion des web services Soap
 - La gestion des web services REST
- Composé d'annotations standards
 - Certaines peuvent nécessiter un framework (en plus du JDK)

JAX-B

20

- JAX-B permet de mapper des objets en XML et inversement (**B**inding)
- Le **B**inding se fait dans les classes Java avec les annotations
- Il ne nécessite pas de framework supplémentaire (depuis Java 6)
 - Attention cependant, ne sera pas opérationnel sur Android par exemple.

JAX-B : Annotations

- **@XmlRootElement** : définit le nom de la balise root de votre flux XML. A définir sur une classe Java.
- **@XmlAccessorType** : définit comment JAX-B doit trouver les balises/attributs. A définir sur une classe Java.
- **@XmlAttribute** : définit l'information comme un attribut de votre balise XML. A définir sur une méthode get/set ou un attribut de classe Java.

JAX-B : Annotations

- **@XmlElement** : définit l'information comme une sous balise de votre balise XML. A définir sur une méthode get/set ou un attribut de classe Java.
- **@XmlElementWrapper** : dans le cas des informations multivaluées (liste, map, tableau), permet de définir une balise parente englobant les informations. A définir sur une méthode get/set ou un attribut de classe Java.
- **@XmlJavaTypeAdapter(UneClasse.class)** : Ou UneClasse hérite de `javax.xml.bind.annotation.adapters.XmlAdapter`, permet de définir comment transformer les informations. A définir sur une méthode get/set ou un attribut de classe Java.

JAX-B : Annotations

➤ Exemples

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<adresse ville="Versailles"
         codePostal="78000"
         pays="France">
  <rue>Pave des gardes</rue>
</adresse>
```

```
package fr.xml;

import javax.xml.bind.annotation.XmlAccessType;
import javax.xml.bind.annotation.XmlAccessorType;
import javax.xml.bind.annotation.XmlAttribute;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;

/**
 * Classe qui represente une adresse. <br/>
 */
@XmlRootElement(name = "adresse")
@XmlAccessorType(XmlAccessType.NONE)
public class Adresse {
    @XmlAttribute
    private String ville;
    @XmlAttribute
    private int codePostal;
    @XmlElement
    private String rue;
    @XmlAttribute
    private String pays;

    /**
     * Constructeur.
     */
    public Adresse() {
        super();
    }
}
```

JAX-B : Fabriquer un flux

- L'écriture se réalise grâce à un **Marshaller**
- L'objet **JAXBContext** se charge d'analyser les classes annotées
 - Attention à ne pas oublier de classes
- De lui on peut obtenir un **Marshaller**

```
// Creation du fichier XML
try {
    JAXBContext jaxbContext = JAXBContext.newInstance(Personne.class, Adresse.class);
    Marshaller jaxbMarshaller = jaxbContext.createMarshaller();
    jaxbMarshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);
    jaxbMarshaller.marshal(adr01, new File(fichierSortie));
} catch (Exception e) {
    e.printStackTrace();
}
```


JAXB : Lire un flux

- La lecture se réalise grâce à un **Unmarshaller**
- L'objet **JAXBContext** se charge d'analyser les classes annotées
 - Attention à ne pas oublier de classes
- De lui on peut obtenir un **Unmarshaller**

```
// Lecture du fichier
try {
    JAXBContext jaxbContext = JAXBContext.newInstance(Personne.class, Adresse.class);
    Unmarshaller jaxbUnmarshaller = jaxbContext.createUnmarshaller();
    Personne personneLue = (Personne) jaxbUnmarshaller.unmarshal(new File(fichierEntree));
    System.out.println(personneLue);
} catch (Exception e) {
    e.printStackTrace();
}
```

JAX-B : JAXBContext

- L'objet **JAXBContext** se charge d'analyser les classes annotées
- Vous pouvez lui indiquer, soit :
 - Les classes annotées séparées par des virgules (**des** objets `java.lang.Class`)
 - Les packages contenant des classes annotées, aussi séparés par des virgules (**un** objet `java.lang.String`)

JAX-B : XmlAdapter

- Quand votre attribut est de type complexe (une **date** par exemple) vous devrez créer un objet qui va expliquer comment faire la transformation XML <-> Valeur
- Cet objet sera une classe Java qui implémentera l'interface **`javax.xml.bind.annotation.adapters.XmlAdapter`**
- Cette classe **générique** prend deux informations
 - ❑ Le type de sortie (en général une String)
 - ❑ Le type de votre attribut

JAX-B : XmlAdapter

- Exemple : ici on a un adapter pour les dates dans le format de notre choix
- <Sortie, Entrée>

```
package fr.xml;

import java.text.SimpleDateFormat;
import java.util.Date;

import javax.xml.bind.annotation.adapters.XmlAdapter;

public class DateAdapter extends XmlAdapter<String, Date> {

    @Override
    public Date unmarshal(String aV) throws Exception {
        if (aV == null || aV.trim().isEmpty()) {
            return null;
        }
        SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
        dateFormat.setLenient(false);
        return dateFormat.parse(aV);
    }

    @Override
    public String marshal(Date aV) throws Exception {
        if (aV == null) {
            return null;
        }
        SimpleDateFormat dateFormat = new SimpleDateFormat("dd/MM/yyyy");
        dateFormat.setLenient(false);
        return dateFormat.format(aV);
    }
}
```

JAX-B : XmlAdapter

- Pour utiliser votre adapter il suffit d'utiliser l'annotation **@XmlJavaTypeAdapter** sur l'attribut ou la méthode get/set

```
@XmlElement  
@XmlJavaTypeAdapter(DateAdapter.class)  
private Date dateNaissance;
```

Travaux pratiques

- Réalisons un Mapping XML de nos entités



JAX-P

31

- L'API Java pour le traitement XML permet aux applications d'analyser et transformer des documents XML en utilisant une API qui est indépendante d'une implémentation d'un processeur XML particulier. (**P**arser)
- JAX-P permet aussi de brancher/débrancher une implémentation de processeur XML simplement
- Toute application qui fait de la gestion de flux XML devrait se limiter à l'API JAXP et éviter l'utilisation d'API spécifiques.
- JAXP sait très bien faire du
 - ❑ DOM : traitement global du flux, notion de chemin et de liens entre balises
 - ❑ SAX : traitement au fur et à mesure

JAX-P

- JAXP sait moins bien faire les transformations XSLT
 - A vous de voir en fonction de vos besoins de transformation
- Avant il fallait ajouter à ses projets XML des librairies comme Xerces ou Xalan
- Depuis le Java 5, une implémentation représentée par JAX-P est disponible dans le JDK
 - StAX

JAX-WS

33

- JAX-WS représente les annotations permettant de réaliser des Web services SOAP
 - Package javax.jws.*
- https://docs.oracle.com/cd/E13222_01/wls/docs103/webser v_ref/annotations.html
- Pour réaliser un projet en SOAP, il faut choisir un framework qui respecte cette norme
 - Apache AXIS / AXIS2
 - Apache CXF
 - gSOAP

JAX-RS

- JAX-RS représente les annotations permettant de réaliser des Web services REST
 - Package `javax.ws.rs.*` ou `jakarta.ws.rs` (JEE 9+)
- <http://docs.oracle.com/javaee/6/tutorial/doc/gilik.html>
- Pour réaliser un projet en REST, il faut choisir un framework qui respecte cette norme
 - Jersey
 - Spring MVC
 - Restlet

JAX-RS

➤ Attention :

- En Java 10+, toutes les APIs JAX sont retirées du JSE
- En Java 10+, il vous faudra ajouter des dépendances vers les API JAX dans votre outil de build

➤ <https://www.oracle.com/technetwork/java/javase/11-relnote-issues-5012449.html#Removed>

Rappel JSON

➤ **JSON** : **Java**Script **O**bject **N**otation

➤ Format de données



- Né dans les années 2002-2005, normalisé par IETF (Internet Engineering Task Force)
- Le JSON sert exclusivement à représenter des données

JSON vs XML

➤ Contrairement à XML en JSON

- Il n'y a pas de balise
- Il n'y a pas d'attribut
- Il n'y a pas de grammaire (DTD, XSD)

➤ Avantages

- Flux plus petit qu'en XML
- Structuration non contraignante et donc simple
- Lu très simplement par les librairies JavaScript

➤ Inconvénients

- Structuration non contraignante et donc non validante
- Type des données limité

Flux JSON

- La structure du JSON se limite à deux représentations
 - Un objet JSON : { ... }
 - Un tableau d'objets JSON : [{...} , {...}, {...}]

- Un objet JSON est composé par
 - Des noms de propriétés entre " " (ou ' ')
 - Des valeurs dont le type peut être :
 - ❑ Chaîne de caractères (entre " " ou ' ')
 - ❑ Nombre (entier ou flottant)
 - ❑ Un objet JSon
 - ❑ Boolean (true ou false)
 - ❑ Un tableau ([])
 - ❑ La valeur null

Exemple : un objet JSON

```
{  
  "lastName": "Gillet",  
  "lastLogin": 1421766774000,  
  "address": {"office": "Etage 1, Porte 281"},  
  "firstLogin": 1421766066000,  
  "sex": 0,  
  "creationDate": 1409750532000,  
  "login": "testuser98"  
}
```

JSON : contraintes syntaxiques

- Le ':' sépare le nom de la propriété de sa valeur
- L'usage des " sur le nom des propriétés est **obligatoire**
 - "lastName" et surtout pas lastName ni 'lastName'
- La ',' permet de séparer chaque propriété entre elles
 - Elle n'est pas obligatoire sur le dernier élément

Exemple un Objet : JSON

- Ceci est un Objet JSON

```
{  
  "employees": [  
    {"firstName":"John", "lastName":"Doe", "sex":true },  
    {"firstName":"Anna", "lastName":"Smith", "sex":true },  
    {"firstName":"Peter","lastName":"Jones", "sex":false }  
  ],  
  "taille" : 3  
}
```

- La valeur de la propriété *employees* est représentée par un **tableau** d'objet JSON.

JSON et Java

- Il n'y a pas pour le moment d'annotation **officielle** comme en JAX-B
- Cependant, vous avez un lien entre JAX-B et JSON
- Vous pouvez faire usage des Marshaller et Unmarshaller de JAX-B afin de produire/lire du JSON
- <https://docs.oracle.com/middleware/1212/toplink/TLADG/json.htm#TLADG1144>

JSON et Java

- L'approche Oracle actuelle est malheureusement assez lourde
- En général, on fait plutôt usage d'un framework extérieur :
 - <https://jsonp.java.net>
 - <http://json-lib.sourceforge.net/>
 - <http://flexjson.sourceforge.net/>
 - <https://github.com/google/gson>
 - <https://github.com/FasterXML/jackson-core>

JSON et Java

➤ Le plus utilisé est **Jackson**

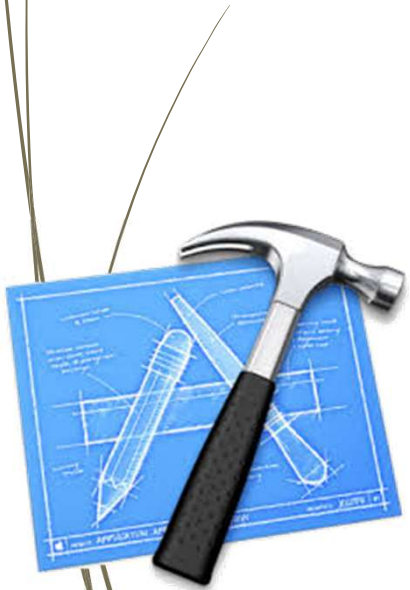
- Le plus rapide
- Possède des annotations (propriétaires)
- Peux aussi faire usage des annotations JAX-B (pratique si vous avez besoin de mapping XML et JSON)
- Fonctionne de manière transparente et automatique avec de nombreux frameworks (Spring, Jersey, ...)
- **MAIS** : ne marche pas sous Android avec pro-guard

Mapping

- En général, le mapping respectera les contraintes suivantes
 - Web service SOAP (en JAX-WS)
 - ❑ XML (via JAX-B)
 - Web Service REST (en JAX-RS) :
 - ❑ XML (via JAX-B)
 - ❑ JSON (via un framework)

Travaux pratiques

- Réalisons un Mapping JSON de nos entités



Web Service : Présentation

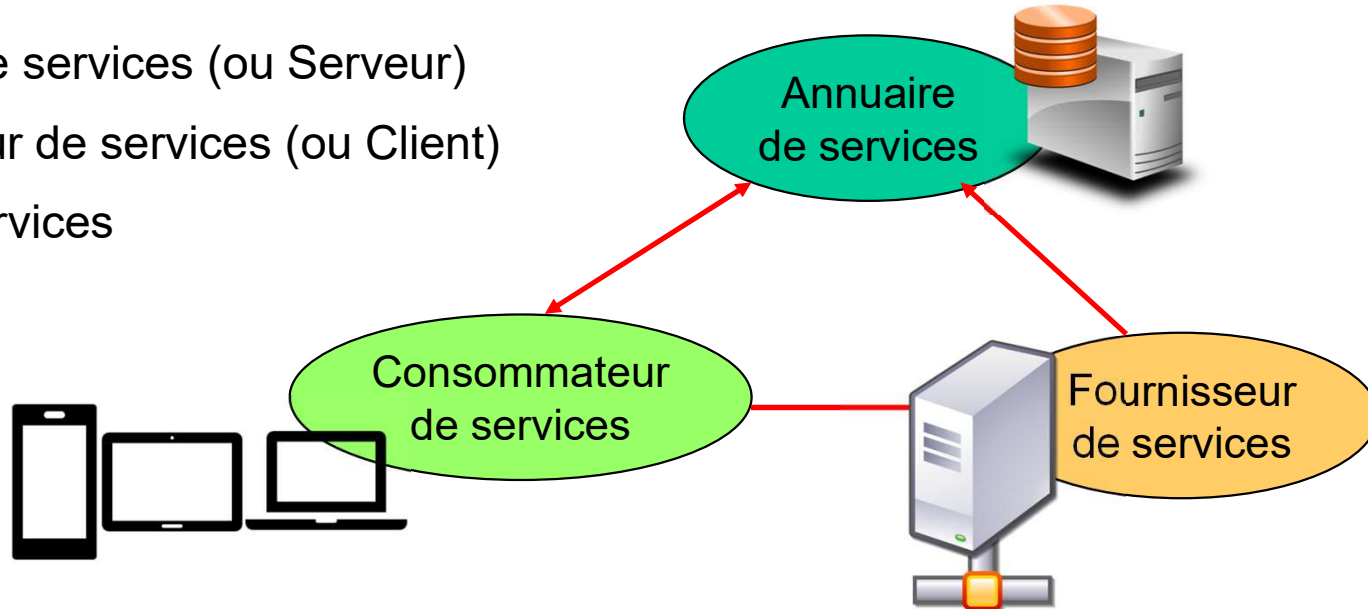
➤ Caractéristiques du service web :

- **Autonome** : L'implémentation d'un service doit être la plus indépendante possible vis-à-vis du parc de services existant.
- **Localisable** : Un service doit être formellement localisable. Dans les standards des services web (WS-*), la localisation se définit à l'aide d'une URI.
- **Isolé** : La logique de communication doit être isolée de la logique métier. Une évolution du protocole de communication ne doit impacter ni la signature du service ni son implémentation.
- **Sans état** : L'invocation courante du service doit être indépendante de son invocation précédente.
- **Transparent** : L'exécution d'un service doit être facilement diagnostiquée (état des variables et des threads, performances ...).

Web Service : Présentation

➤ Architecture et rôle

- Les Web services sont construits sur le concept de l'informatique distribuée
- Une architecture Web Services consiste en trois rôles ou environnement d'exécution
 - ❑ Le fournisseur de services (ou Serveur)
 - ❑ Le consommateur de services (ou Client)
 - ❑ L'annuaire de services



Web Service : Présentation



➤ Le fournisseur de services (ou Serveur)

- Il exécute le Web Service et fournit l'accès à celui-ci au travers d'une interface standard
- Il est séparé en deux entités
 - ❑ L'interface du service : elle fournit le contrat du service (ce qu'on peut utiliser) aux applications utilisatrices
 - ❑ L'implémentation du service : elle fournit l'implémentation de cette interface
 - Exemple : un composant hébergé dans un serveur d'application

Web Service : Présentation



- Le consommateur de services (ou Client)
 - Il s'agit de l'application utilisatrice du Web Service
 - Il établit une liaison avec l'interface du service proposée par le fournisseur de services
 - Puis il utilise le service souhaité

- Un consommateur est donc un programme qui sait envoyer des requêtes HTTP et recevoir des flux

Web Service : Présentation



➤ L'annuaire de services

➤ Il fournit le moyen de trouver les Web Services publiés par les fournisseurs de services

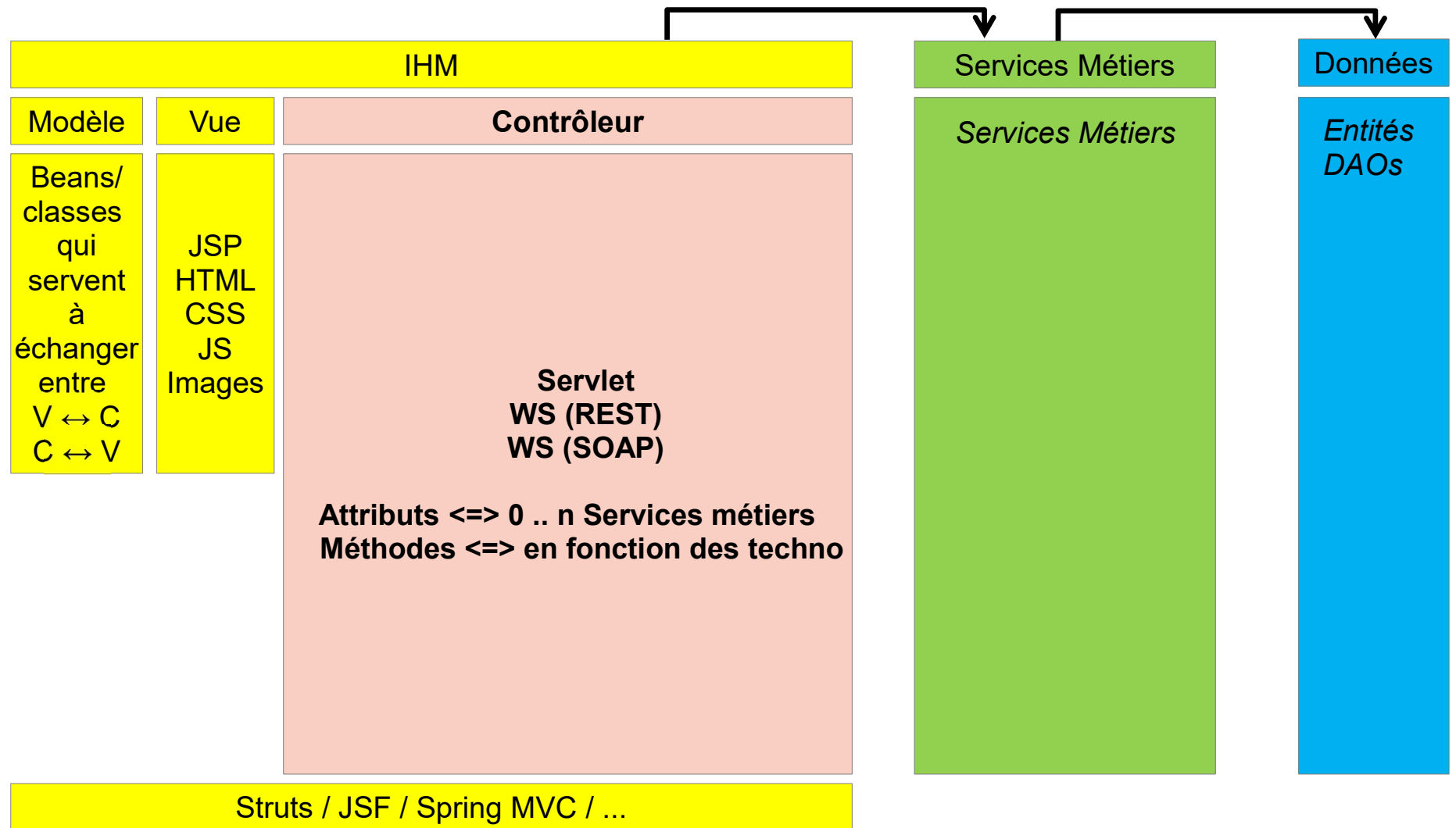
➤ Principes

- ☐ Le fournisseur de services publie ses services dans l'annuaire
- ☐ Le consommateur de services s'adresse à l'annuaire pour obtenir des informations sur les services disponibles

➤ En Spring

- Eureka
- HashiCorp Consul

Web Service : Présentation

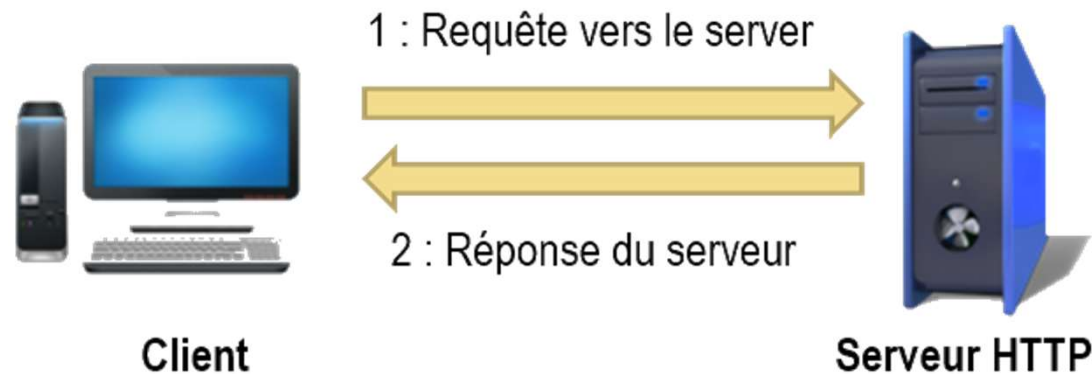


Le protocole HTTP

➤ Un protocole est une convention à respecter pour bien communiquer.

➤ HTTP :

- ❑ **H**yper**T**ext **T**ransfer **P**rotocol soit protocole de transfert hypertexte
- ❑ Développé pour le World Wide Web
- ❑ Il fait partie de la couche "application"
- ❑ Son port par défaut est le port 80 (443 pour le https)



HTTP : Communication

- Le HTTP permet d'échanger des flux textuels entre un client et un serveur
- Le client fait une requête (= une demande) au serveur.
Ces demandes peuvent être de plusieurs types utilisant ce que l'on appelle "méthodes" / "Commandes"
 - **get** : Demander une ressource au serveur (exemple: un fichier, une page html ou des informations).
 - **post** : Envoyer des données vers le serveur (exemple : enregistrer un formulaire, envoyer un message)
 - **put** : Créer ou modifier une ressource
 - **delete** : Supprimer des ressources
 - **head** : Demande d'information sur une ressource.
- Le serveur exécute la demande et répond
 - Un flux
 - Un HTTP Status

Le protocole HTTP : requête

- La requête est divisée en deux blocs :
- L'entête qui contiendra des informations sur la méthode employée, le serveur et le client lui-même

En-tête GET

```
GET /chemin/hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.site.com
```

En-tête POST

```
POST /chemin/connexion HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.site.com
```

- Le corps contient les données envoyées

Corps POST

```
Login: John
Password: 1234
```

HTTP : GET

- C'est la méthode la plus courante pour demander une ressource.
- Une requête **GET** est sans effet sur la ressource, elle ne modifie pas cette dernière
- On peut utiliser le **GET** quand on souhaite réaliser un select, insert, update, delete
- On fait un **GET** quand
 - On clique sur un lien hypertexte
 - On soumet un formulaire qui est en méthode GET
 - Dans ce cas, les paramètres **seront visibles** sur l'URL
- **Attention** : Vous ne pourrez pas utiliser @RequestBody sur du GET
 - Donc pas d'envoi de JSON en GET

HTTP : POST

- Cette méthode est utilisée pour transmettre des données en vue d'un traitement à une ressource.
- Une requête **POST** entraîne généralement un traitement côté serveur.
- On peut utiliser le **POST** quand on souhaite réaliser un select, insert, update, delete
- On fait un **POST** quand
 - On soumet un formulaire qui est en méthode POST
 - Dans ce cas, les paramètres **ne seront pas visibles** sur l'URL

HTTP : PUT

- Cette méthode permet de remplacer ou d'ajouter une ressource sur le serveur.
- On peut utiliser le **PUT** quand on souhaite réaliser un insert, update, delete
- Le **PUT** a deux avantages par rapport aux POST/GET
 - Son résultat ne sera pas mis en cache
 - Si deux requêtes PUT identiques arrivent sur le serveur, une seule sera traitée (idempotent)
- On ne peut faire du **PUT** qu'à travers du code

HTTP : PATCH

- Cette méthode permet, contrairement à PUT, de faire une modification partielle d'une ressource.
- On peut utiliser le **PATCH** quand on souhaite réaliser un update
- On ne peut faire du **PATCH** qu'à travers du code
- Attention :
 - **PATCH** n'est pas idempotent
 - N'est pas supporté par tous les serveurs / framework

HTTP : DELETE

- Cette méthode permet de supprimer une ressource du serveur.
- On peut utiliser le **DELETE** quand on souhaite réaliser un delete
- On ne peut faire du **DELETE** qu'à travers du code

HTTP : OPTIONS

- Cette méthode permet d'obtenir les options de communication d'une ressource ou du serveur en général.
- On peut utiliser le **OPTIONS** quand on souhaite récupérer des informations
- On ne peut faire du **OPTIONS** qu'à travers du code

HTTP : TRACE

- Cette méthode demande au serveur de retourner ce qu'il a reçu, dans le but de tester et effectuer un diagnostic sur la connexion.
- On peut utiliser le **TRACE** quand on souhaite récupérer des informations
- On ne peut faire du **TRACE** qu'à travers du code

HTTP : CONNECT

- Cette méthode permet d'utiliser un proxy comme un tunnel de communication.
- On ne peut faire du **CONNECT** qu'à travers du code

HTTP : HEAD

- Cette méthode ne demande que des informations sur la ressource, sans demander la ressource elle-même.
- On ne peut faire du HEAD qu'à travers du code

Méthodes HTTP Récapitulatif

Method	Idempotent	Safe
OPTIONS	Oui	Oui
GET	Oui	Oui
HEAD	Oui	Oui
PUT	Oui	Non
DELETE	Oui	Non
PATCH	Non	Non
POST	Non	Non

Méthodes HTTP

- Une méthode **idempotente** signifie que le résultat d'une requête effectuée avec succès est indépendant du nombre de fois où elle a été exécutée.
 - OPTIONS, GET, HEAD, PUT, DELETE sont *idempotentes*
- Une méthode **safe** signifie que la requête ne modifie en rien l'état du serveur.
 - OPTIONS, GET, HEAD sont *safe*

REST

- **RE**presentational **S**tate **T**ransfer
- Moyen de demander des traitements ou informations à un serveur
- Repose sur le protocole HTTP (et ses méthodes GET, PUT, ...)
- En théorie sans état (en pratique cela se complique)
 - Chaque requête est indépendante

REST sans état

- Le client demande quelque chose au serveur via une commande une URI et des paramètres
 - Paramètres WEB (GET classique)
 - Paramètres dans le HEAD ou le BODY
- Le serveur analyse sa demande et répond sous la forme d'un flux (XML / JSON / Binaire) et/ou d'un statut
- Le client et le serveur ne se connaissent plus

REST et HTTP

- Lors de la réalisation de mécanique CRUD :
 - Create = **PUT**
 - Retrieve = **GET**
 - Update = **PUT** (ou PATCH)
 - Delete = **DELETE**
- On évite le POST (et souvent le PATCH) sur les actions d'écriture de données car il n'est pas *idempotent*
- On parle ici de recommandations, vous pouvez tout faire en GET si cela vous chante, c'est à **vous** de décider de la méthode HTTP à utiliser

REST et URL

- Vos Web services seront disponibles sur des URL
- Pour passer de l'information vous avez trois techniques :
 1. Passer des paramètres : URL?clef=valeur&clef=valeur ...
 2. Passer des informations dynamiques sur l'URL : URL/valDynA/valDynB
 3. Passer des objets dans le body lors de l'appel (en JSon ou autre)
- En fonction de vos besoins et des frameworks, vous pouvez mixer les trois techniques
 - ❑ Ajax simple : technique 1, 2, 3
 - ❑ Angular : technique 2, 3
 - ❑ Web simple : technique (via formulaire) 1, 2

REST et URL

71

- Exemple, imaginons que nous réalisons des web services REST sur la gestion de livres.
 - On ne parle ici que des techniques 1, 2 car la troisième n'est pas visible
- Rechercher un ou plusieurs livres (en **GET**)
 - *http://serveur/context/rest/livres?crt0=val0&crt1=val1*
 - Ex: *http://serveur/context/rest/livres?titre=terre&annee=1986*
- Supprimer un livre (en **DELETE**)
 - *http://serveur/context/rest/livres/valeurIdDuLivre*
 - Ex : *http://serveur/context/rest/livres/109*

REST et URL

➤ Mettre à jour un livre (en PATCH ou PUT)

- `http://serveur/context/rest/livres/valldDuLivre?col0=val0&col3=val3`
- Ex : `http://serveur/context/rest/livres/109?annee=2008&nbpage=298`

➤ Insérer un livre (en PUT)

- `http://serveur/context/rest/livres?col0=val0&col1=val1`
- Ex :
`http://serveur/context/rest/livres?titre=S4&author=Max&annee=2016&nbpage=405`

REST et URL

73

➤ Même exemple, mais via le 2,3 (utilisation du JSON)

➤ On définit le flux JSON suivant qui représentera un livre :

➤ On définit le flux JSON suivant qui représentera des livres :

```
{
  "id" : 103,
  "titre" : "xxxx",
  "annee" : 2014,
  "nbpage" : 50,
  "author" : "Jhon Smith"
}
```

```
{
  taille : 2,
  livres : [{
    "id" : 103,
    "titre" : "xxxx",
    "annee" : 2014,
    "nbpage" : 50,
    "author" : "Jhon Smith"
  },
  {
    "id" : 104,
    "titre" : "xxxx",
    "annee" : 2015,
    "nbpage" : 150,
    "author" : "Jhon Smith"
  }
  ]
}
```

REST et URL

- Rechercher un ou plusieurs livres (en **PUT**, et pas en ~~GET~~ car pas de body)

- *http://serveur/context/rest/livres*

- Ex: *http://serveur/context/rest/livres*

- ❑ On passe dans le **body** un objet JSON de recherche qui ressemble à l'objet JSON livre mais où toutes les propriétés sont optionnelles

- ❑ On récupère un objet JSON de type livres

- Supprimer un livre (en **DELETE**)

- *http://serveur/context/rest/livres/valeurIdDuLivre*

- Ex : *http://serveur/context/rest/livres/109*

REST et URL

75

➤ Mettre à jour un livre (en **PATCH** ou **PUT**)

➤ *http://serveur/context/rest/livres/valldDuLivre*

➤ Ex : *http://serveur/context/rest/livres/109*

- ☐ On passe dans le **body** un objet JSON de type livre mais où toutes les propriétés sont optionnelles
- ☐ On récupère un objet JSON de type livre

➤ Insérer un livre (en **PUT**)

➤ *http://serveur/context/rest/livres*

➤ Ex : *http://serveur/context/rest/livres*

- ☐ On passe dans le **body** un objet JSON de type livre
- ☐ On récupère un objet JSON de type livre (avec son ID)

HTTP : Réponse

- La réponse tout comme la demande est composée elle aussi des deux parties, l'en-tête et le corps.

En-tête réponse get

```
HTTP/1.1 200 OK  
Date: Fri, 06 May 2016 16:04:09 GMT  
Server: Apache/2.4.6 (CentOS) PHP/5.4.16
```

Corps réponse get

```
<!DOCTYPE html>  
<head>  
<meta charset="utf-8">  
</head>  
<body>  
...
```

REST et les codes retours - status

- Quand le serveur répond, il répond un status et potentiellement un flux
- Codes commençant par :
 - **1xx** : Information pour le client.
 - **2xx** : Indique que la demande a été traitée avec succès
 - **3xx** : Indique qu'il faut rediriger le client ou que la ressource a été déplacée.
 - **4xx** : Généralement une erreur du client
 - **5xx** : Le serveur a rencontré un problème de fonctionnement
- https://fr.wikipedia.org/wiki/Liste_des_codes_HTTP

REST et les codes retours - status

- **ATTENTION** : selon qui a fait le demande (code Java, Javascript, client web, ...) **si** le code retour n'est pas 200 alors il faudra tester le comportement
- Un client peut traiter le body d'une réponse 500 alors qu'un autre ne le fera pas.
 - Ne soyez pas trop *créatif* sur l'usage des *status*

Problématique de l'authentification

- Si un web service REST ne conserve pas d'information, comment gérer l'authentification ?
- Scénario classique (en MVC - Mode JSP/Servlet)
 - ❑ Vue login : soumission du formulaire web qui nous fait partir vers le contrôleur d'authentification
 - ❑ Contrôleur d'authentification : Il valide les paramètres, fait l'authentification, place en session une information puis passe à l'écran suivant

Problématique de l'authentification : S1

- Le web service d'authentification renvoie une information (un id, un token d'authentification, ...) dans le body de réponse.
- Ce token est conservé par le client et passé dans toutes les requêtes suivantes en tant que paramètre
 - `http://serveur/context/rest/livres?token=Xxx&col0=val0&col1=val1`
- **Avantage**
 - Très simple à développer
 - On reste en REST (sans état)
- **Inconvénient**
 - Lourdeur dans la gestion des URI

Problématique de l'authentification : S2

- On fait usage de l'objet HttpSession dans le web service côté serveur.
- On fait comme si le web service REST était une servlet
- **Avantage**
 - Très simple à développer
- **Inconvénient**
 - On ne respecte pas la norme REST (sans état)
 - Certains clients peuvent perdre l'id de session ou ne pas le renvoyer

Problématique de l'authentification : S3

- C'est la solution 1, mais au lieu de placer le token dans l'URL ou le body on le place dans le Header HTTP
- **Avantage**
 - On respecte la norme REST (sans état)
- **Inconvénient**
 - Pas toujours simple à développer

Problématique de l'authentification : SX

- En fonction des implémentations de framework vous pourrez faire usage de solutions automatiques.
- Par exemple <https://jwt.io/> (JsonWebToken)
 - Disponible dans Spring Security
- Restez simple et testez bien votre choix sur plusieurs serveurs et plusieurs clients (navigateurs, clients JS)

Problématique de documentation

- Faire des web services c'est bien, mais il n'y pas de moyen autre que la documentation pour savoir :
 - Ce que l'on doit lui envoyer
 - ❑ paramètres, JSON, format du JSON
 - ❑ méthode HTTP
 - Ce que l'on va recevoir
 - ❑ Flux JSON
 - ❑ Status HTTP

Problématique de documentation

- Vous pouvez faire usage de framework pour générer automatiquement et dynamiquement de la documentation sur vos web services
- Par exemple via
 - Le framework **Swagger** : <https://swagger.io/>
 - Ou si vous êtes en Spring 5.2+ **Spring-OpenAPI** : <https://springdoc.github.io/springdoc-openapi-demos/>
- **Attention** : sa mise en place peut demander du temps

Problématique de documentation

➤ Exemple d'ajout de la dépendance en Spring Boot

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <!-- https://mvnrepository.com/artifact/org.springdoc/springdoc-openapi-starter-webmvc-ui -->
  <!-- ATTENTION aux versions -->
  <version>3.0.3</version>
</dependency>
```

- L'ajout de cette dépendance suffit à vous fournir dynamiquement une page web de documentations sur votre API Rest
 - ➡ Disponible sur <http://xxxxx/votrecontext/swagger-ui.html>

Problématique de documentation

➤ Si vous voulez configurer / paramétrer votre Swagger

➤ Les annotations

- ❑ `@io.swagger.v3.oas.annotations.tags.Tag` : sur son contrôleur
- ❑ `@Operation` : sur ses méthodes de contrôleur afin de décrire le end-point
- ❑ `@ApiResponse` : sur ses méthodes de contrôleur afin de décrire les http status et flux de retours possibles
- ❑ `@Parameter` : sur les paramètres de vos end-point pour décrire chacun d'eux
- ❑ `@Schema` : sur les classes de DTO et leurs attributs afin de décrire les modèles de flux JSON

Problématique de documentation

➤ Exemple

```
@PostMapping("/add")
@Operation(tags = {"Constraint management API" },
    summary = "Adds a constraint.", description = "Will add a constraint into the data base.",
    security = { @SecurityRequirement(name = "bearer-key") })
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Your constraint was added and returned in the response body.",
        content = @Content(schema = @Schema(implementation = ConstraintDtoOut.class))),
    @ApiResponse(responseCode = "400", description = "Your constraint is not valid.",
        content = @Content(schema = @Schema(implementation = ExceptionDtoOut.class))),
    @ApiResponse(responseCode = "401", description = "You are not connected or do not have the LunchLady role.",
        content = @Content(schema = @Schema(implementation = ExceptionDtoOut.class))) })
public ResponseEntity<ConstraintDtoOut> addConstraint(
    @Parameter(description = "Constraint object that will be stored in database.", required = true) @RequestBody ConstraintDtoIn pConstraint) {
    // ...
}
```


Problématique de documentation

- Selon le niveau de complexité de votre API
 - Vous pouvez aussi créer une classe de configuration dédiée

- **ATTENTION**
 - Pensez à sécuriser l'accès à votre swagger
 - Potentiellement la désactiver en production

Problématique du Mapping JSON

- Dans l'approche classique REST
 - ➡ Un WS renvoie un flux JSON
- Par exemple, je cherche l'utilisateur ayant l'ID 1
 - ➡ <https://.../user/1> en GET : me retourne le flux JSON associé à l'objet déclaré dans mon code

```
{  
  "id" : 1,  
  "nom" : "Dupont",  
  "prenom" : "Jean",  
  "age" : 50,  
  ...  
}
```

Problématique du Mapping JSON

- L'objet retourné c'est quoi ?
 - Une Entité ?
 - Un POJO ?
 - Une String ?

- Le choix ne sera pas toujours simple et sera dépendant de vos projets.

Problématique du Mapping JSON

- Une **entité** (un objet qui représente la base de données)
 - **Non** : car, une Entité = n'est PAS un flux à destination ou en provenance du 'client'
 - **Non** : car il n'y a pas de point commun entre un flux et une base de données
 - ❑ Sauf dans le cas des bases NoSQL
 - **Non** : car il ne doit pas y avoir de contraintes JSON dans une entité
 - **Oui** : car de cette manière il n'y a rien d'autre à coder en plus

Problématique du Mapping JSON

- Un **POJO**, on leur donne le nom de **DTO** (Data Transfer Object) :
 - **Oui** : car il sera exactement comme on le souhaite, avec les propriétés et le typage désiré
 - **Oui** : car il sera simple à maintenir
 - **Oui** et Non : car si j'ai besoin de plusieurs flux représentant un Utilisateur je devrais faire plusieurs objets POJO
 - **Non** : car si je dois récupérer 1 million d'utilisateurs = je vais instancier 1 million de POJO pour avoir ensuite un flux JSON

Problématique du Mapping JSON

- En Spring, par défaut, c'est le framework **Jackson** qui s'occupe de gérer les transformations en JSON
 - <https://github.com/fasterxml/jackson>
- Si vous êtes en Spring Boot, vous avez automatiquement un @Bean **ObjectMapper** / **JsonMapper** qui est disponible pour vous
 - Vous n'avez qu'à l'injecter pour jouer avec
 - JsonMapper hérite de ObjectMapper (c'est toujours du Jackson)
- **ATTENTION**
 - Spring Boot < 4 : Jackson 2 ⇔ **ObjectMapper**
 - Spring Boot >= 4 : Jackson 3 ⇔ **JsonMapper**

Problématique du Mapping JSON

- Si vous ne voulez pas de **Jackson**, mais par exemple préférez **Gson**
 - Indiquez dans votre fichier properties Spring Boot
 - ☐ `spring.mvc.converters.preferred-json-mapper=gson`
 - Dans votre Maven / Gradle
 - ☐ Excluez les dépendances Jackson
 - ☐ Incluez la dépendance Gson
- Dans ce cas de figure, vous n'avez pas d'ObjectMapper/JsonMapper de disponible (=Jackson)

Problématique du Mapping JSON

- Dans le cas du **POJO**, vous pouvez
 - Créer vous-même vos classes qui seront responsables de transformer entité -> DTO et/ou DTO -> entité
 - Faire usage du framework MapStruct
 - ❑ <https://mapstruct.org/>
 - Faire usage de ModelMapper
 - ❑ <https://modelmapper.org/>



Problématique du Mapping JSON

- Si vous devez reprendre totalement la main sur la sérialisation JSon, vous avez plusieurs options
- En Jackson 2 (Spring Boot < 4), vous ferez usage des classes abstraites **StdSerializer** / **StdDeserializer** :
 - ➡ A partir de Spring Boot 4, vous êtes en Jackson 3
 - ❑ Il est encore possible de faire du Jackson 2, mais il est fortement conseillé de migrer
- Vous utiliserez les annotations **@JsonSerialize** et **@JsonDeserialize** pour les monter sur le DTO visé

Problématique du Mapping JSON

- A partir de Spring Boot 4 (Jackson 3), vous avez deux approches
 - Vos classes hériteront des classes abstraites **ValueSerializer** / **ValueDeserializer** (vous êtes en Jackson)
 - ❑ Vous avez une adhérence sur Jackson
 - ❑ Vous pouvez continuer à faire usage de **@JsonSerialize** / **@JsonDeserialize**
 - Ou, elles hériteront des classes abstraites **ObjectValueSerializer** / **ObjectValueDeserializer** (vous êtes en Spring)
 - ❑ Vous n'avez pas d'adhérence sur Jackson
- Pour qu'elles soient montées automatiquement vous pouvez les annoter avec **@JacksonComponent**

Problématique du Mapping JSON

➤ Exemple en Spring Boot < 4 / Jackson 2 (StdSerializer)

```
public class PhonesJsonSerializer extends StdSerializer<PhonesJson> {

    public PhonesJsonSerializer() {
        this(PhonesJson.class);
    }

    public PhonesJsonSerializer(Class<PhonesJson> pClass) {
        super(pClass);
    }

    @Override
    public void serialize(PhonesJson pPhones, JsonGenerator jsonGenerator, SerializerProvider serializer) throws IOException {
        jsonGenerator.writeStartObject();
        for (Map.Entry<String, String> entry : pPhones) {
            jsonGenerator.writeStringField(entry.getKey(), entry.getValue());
        }
        jsonGenerator.writeEndObject();
    }
}
```

```
@JsonSerialize(using = PhonesJsonSerializer.class)
@JsonDeserialize(using = PhonesJsonDeserializer.class)
public class PhonesJson implements Serializable, Iterable<Map.Entry<String, String>> {
    // ..
}
```

Problématique du Mapping JSON

➤ Exemple en Spring Boot < 4 / Jackson 2 (StdDeserializer)

```
public class PhonesJsonDeserializer extends StdDeserializer<PhonesJson> {  
    public PhonesJsonDeserializer() {  
        this(PhonesJson.class);  
    }  
  
    public PhonesJsonDeserializer(Class<PhonesJson> pClass) {  
        super(pClass);  
    }  
  
    @Override  
    public PhonesJson deserialize(JsonParser parser, DeserializationContext deserializer) throws IOException {  
        PhonesJson result = new PhonesJson();  
        ObjectCodec codec = parser.getCodec();  
        JsonNode node = codec.readTree(parser);  
  
        Iterator<String> iter = node.fieldNames();  
        while (iter.hasNext()) {  
            String name = iter.next();  
            result.addPhone(name, node.get(name).asText());  
        }  
  
        return result;  
    }  
}
```

```
@JsonSerialize(using = PhonesJsonSerializer.class)  
@JsonDeserialize(using = PhonesJsonDeserializer.class)  
public class PhonesJson implements Serializable, Iterable<Map.Entry<String, String>> {  
    // ..  
}
```

Problématique du Mapping JSON

➤ Exemple en Spring Boot >= 4 / Jackson 3 (ValueSerializer)

```
@JacksonComponent // Ou @JsonSerialize sur le bean visé
public class PhonesJsonSerializer extends ValueSerializer<PhonesJson> {

    @Override
    public void serialize(PhonesJson phones, JsonGenerator jsonGenerator, SerializationContext context)
        throws IOException {

        jsonGenerator.writeStartObject();
        for (Map.Entry<String, String> entry : phones) {
            jsonGenerator.writeStringField(entry.getKey(), entry.getValue());
        }
        jsonGenerator.writeEndObject();
    }
}
```

Problématique du Mapping JSON

➤ Exemple en Spring Boot >= 4 / Jackson 3 (ValueDeserializer)

```
@JacksonComponent // Ou @JsonDeserialize sur le bean visé
public class PhonesJsonDeserializer extends ValueDeserializer<PhonesJson> {

    @Override
    public PhonesJson deserialize(JsonParser parser, DeserializationContext context) throws IOException {
        PhonesJson result = new PhonesJson();
        ObjectCodec codec = parser.getCodec();
        JsonNode node = codec.readTree(parser);

        Iterator<String> iter = node.fieldNames();
        while (iter.hasNext()) {
            String name = iter.next();
            result.addPhone(name, node.get(name).asText());
        }

        return result;
    }
}
```

Problématique du Mapping JSON

➤ Exemple en Spring Boot >= 4 / Spring (ObjectValueSerializer)

@JacksonComponent

```
public class PhonesJsonSerializer extends ObjectValueSerializer<PhonesJson> {
```

```
    @Override
```

```
    protected void serializeObject(PhonesJson phones, JsonGenerator jsonGenerator, SerializationContext context)
                                                                    throws IOException {
```

```
        for (Map.Entry<String, String> entry : phones) {
            jsonGenerator.writeStringField(entry.getKey(), entry.getValue());
        }
    }
```

```
}
```

Problématique du Mapping JSON

- Exemple en Spring Boot >= 4 / Spring (ObjectValueDeserializer)

@JacksonComponent

```
public class PhonesJsonDeserializer extends ObjectValueDeserializer<PhonesJson> {
```

```
    @Override
```

```
    protected PhonesJson deserializeObject(JsonParser parser, DeserializationContext context, JsonNode node)
        throws IOException {
```

```
        PhonesJson result = new PhonesJson();
```

```
        Iterator<String> iter = node.fieldNames();
```

```
        while (iter.hasNext()) {
```

```
            String name = iter.next();
```

```
            result.addPhone(name, node.get(name).asText());
```

```
        }
```

```
        return result;
```

```
    }
```

```
}
```

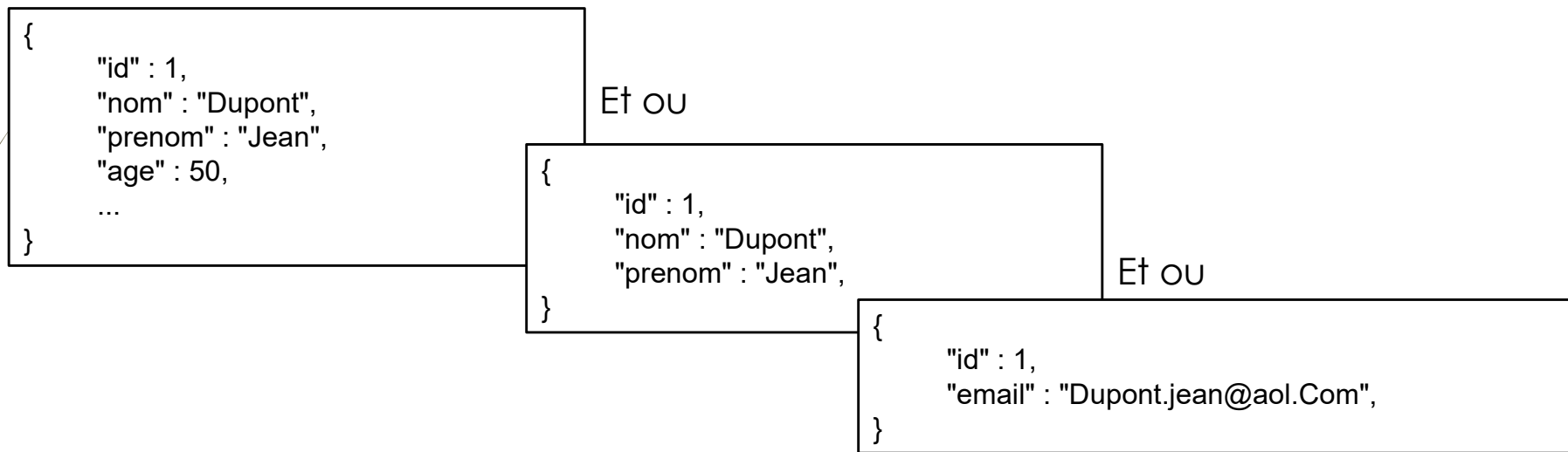

Problématique du Mapping JSON

- Et si je dois faire un flux Json avec une classe qui n'est pas à moi (dans mon projet) ?
 - Vous pouvez utiliser l'annotation `@JacksonMixin`
- Dans cet exemple, *AClassThatIsNotMine* n'est pas modifiable mais j'ai besoin de réaliser un flux JSON sans son attribut name

```
@JacksonMixin(AClassThatIsNotMine.class)
public abstract class MyMappingForThisExternalClass {
    @JsonIgnore
    abstract String getName();
}
```

Problématique du Mapping JSON

- Et si je dois avoir plusieurs représentations d'un même flux ?



Problématique du Mapping JSON

➤ Plusieurs options :

➡ Avoir plusieurs POJO / DTO

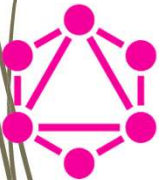
❑ Peut mener à : UtilisateurDtoA, UtilisateurDtoB, ... associés à leur *Mapper*

➡ Avoir plusieurs xxSerializer/Deserializer

❑ Même problème que plusieurs POJO

➡ Utiliser l'annotation **@JsonView**

➤ Dans le cas où vous avez besoin d'une représentation DYNAMIQUE de vos flux, une bonne approche est de faire usage de **GraphQL**.



Problématique du Mapping JSON

108

➤ Avec @JsonView:

- Votre mapping JSON se fera via **un seul DTO**
- Généralement, dedans vous allez créer des inner interfaces vides qui serviront de label/View
- Sur les attributs vous indiquerez @JsonView accompagné de la classe de votre inner interface

```
public class UserDto {  
  
    // Définitions de vos 'View' sous la forme d'inner interface  
    public interface Public {}  
    public interface Admin {}  
  
    @JsonView(Public.class)  
    private Long id;  
  
    @JsonView(Public.class)  
    private String username;  
  
    @JsonView(Admin.class)  
    private String email;  
  
    @JsonView(Admin.class)  
    private String internalNote;  
  
    // Get et Set  
  
}
```

Problématique du Mapping JSON

109

➤ Avec @JsonView:

- Si vous voulez qu'un attribut fasse partie de plusieurs View, il faudra jouer avec l'héritage de ces dernières

```
public class UserDto {  
  
    // Définitions de vos 'View' sous la forme d'inner interface  
    public interface Public {}  
    public interface Admin {}  
    public interface Both extends Admin, Public {}  
  
    @JsonView(Both.class)  
    private Long id;  
  
    @JsonView(Both.class)  
    private String username;  
  
    @JsonView(Admin.class)  
    private String email;  
  
    @JsonView(Admin.class)  
    private String internalNote;  
  
    // Get et Set  
  
}
```

Problématique du Mapping JSON

➤ Avec @JsonView:

- Dans vos contrôleurs, quand vous retournerez un flux, vous indiquerez aussi la View que vous désirez

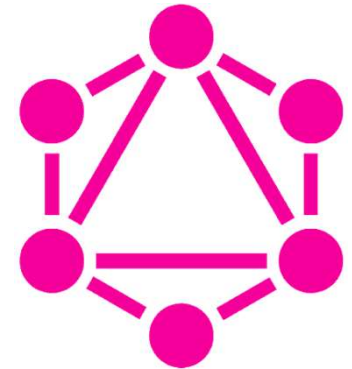
```
@GetMapping("/users/abc")
@JsonView(UserDto.Both.class)
public UserDto getAbc() {
    // ...
}

@GetMapping("/users/def")
@JsonView(UserDto.Admin.class)
public UserDto getDef() {
    // ..
}
```

Problématique du Mapping JSON

➤ En GraphQL :

- Votre mapping JSON se fera via un seul DTO
- Vous l'accompagnez d'un *schema*
 - ❑ Fichier de mapping descriptif
 - ❑ Qui peut contenir des requêtes



- c'est au 'client' d'exprimer son flux désiré lors de la demande

Demande du client

```
{
  userById(id: 1) {
    id
    nom
    prenom
    age
  }
}
```

Demande du client

```
{
  userById(id: 1) {
    id
    nom
    prenom
  }
}
```

Demande du client

```
{
  userById(id: 1) {
    id
    email
  }
}
```

Problématique du Mapping JSON



- En **GraphQL**, le client demande via une requête GraphQL
 - En Get : porte le nom et le flux désiré de retour
 - En Post : porte plus d'informations dont un body
- GraphQL et Spring : Les contrôleurs
 - Sont de simples **@Controller**
 - Ne portent **plus** de mapping sur les méthodes (GET / POST / PUT)
 - Ne portent plus de mapping sur les routes
- Spring sait gérer le GraphQL sans problème, à vous de valider que votre 'client' en est capable

Problématique du Mapping JSON



➤ Exemple : un contrôleur en GraphQL

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.graphql.data.method.annotation.Argument;
import org.springframework.graphql.data.method.annotation.QueryMapping;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Controller;

// ...

@Controller
public class UserRestController {

    @Autowired
    private UserService service;

    @QueryMapping // Implique que la méthode userById est décrite dans le fichier schema de GraphQL
    public UserDto userById(@Argument(name = "id") Integer pUserId) throws EntityNotFoundException {
        var result = this.service.find(pUserId); // On récupère une entité
        return new UserDto(result); // On la transforme en Dto
    }
}
```



Problématique du Mapping JSON

114

Select

➤ Exemple : fichier de mapping/schema GraphQL

```
type Query {  
  userById(id: ID!): UserDtoOut  
}
```

```
type Mutation {  
  registerUser(user: UserDtoIn!): UserDtoOut  
}
```

```
enum Sex {  
  MAN  
  WOMAN  
  OTHER  
}
```

```
type UserDtoOut {  
  id: ID!  
  address: String  
  registrationDate: String  
  email: String!  
  name: String  
  firstname: String  
  sex: Sex  
}
```

insert/update/delete

WebSocket

```
type Subscription {  
  userInfo (id: ID!): UserDtoOut  
}
```

```
type UserDtoIn {  
  address: String  
  registrationDate: String  
  email: String!  
  pwd: String!  
  name: String  
  firstname: String  
  sex: Sex  
}
```

Fichier : src\main\resources\graphql\schema.graphqls

GraphQL en Spring côté contrôleurs

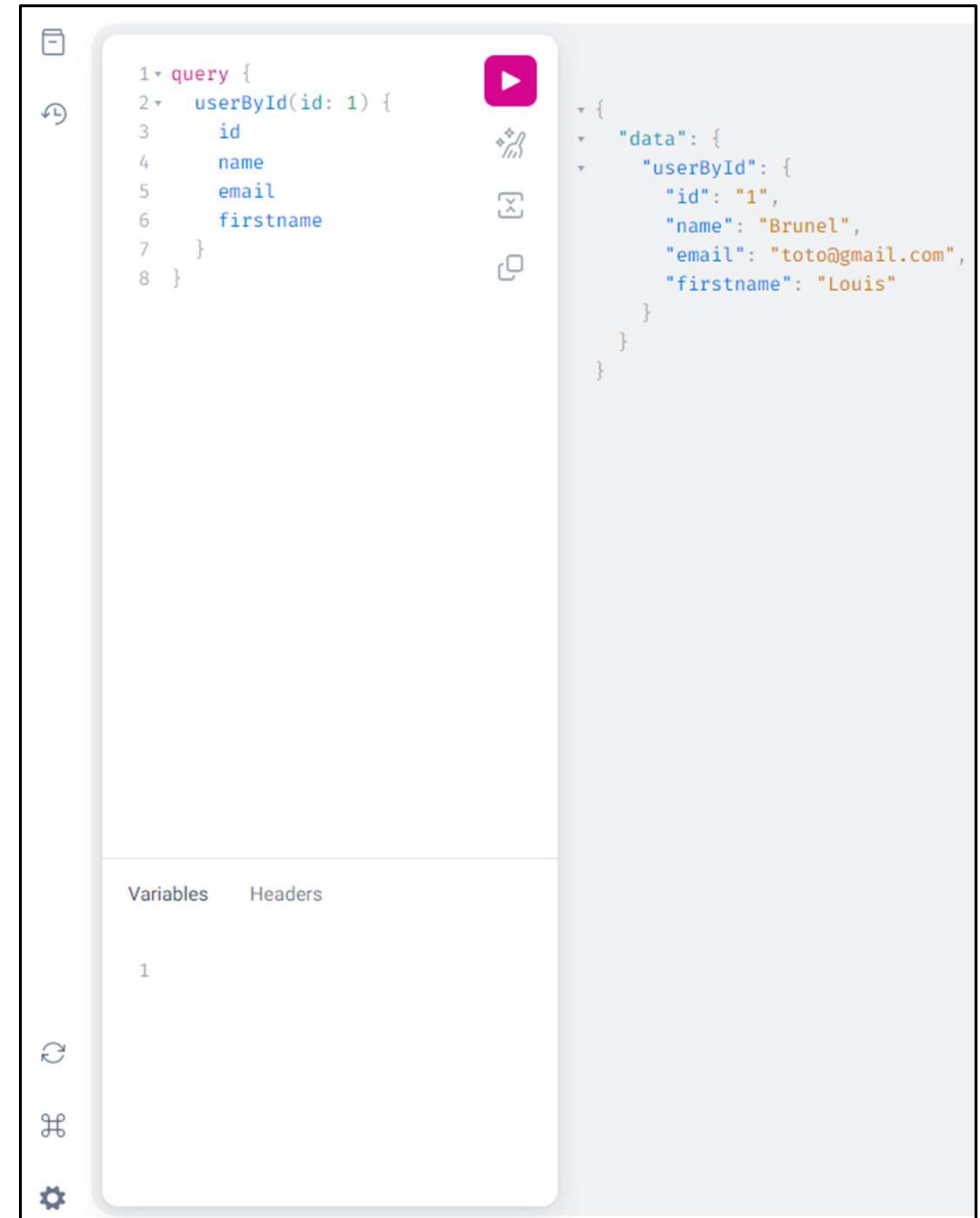


- Supporte plusieurs modes de fonctionnement
 - **@QueryMapping** : pour les requêtes de style select
 - **@MutationMapping** : pour les requêtes de style insert/update/delete
 - **@SubscriptionMapping** : pour les requêtes qui vont pusher l'information au client

- Les **SubscriptionMapping** feront usage des **WebSockets** ou **RSocket**.
 - C'est au front de gérer cette problématique

GraphQL en Spring

- Le Spring fournit, au besoin, une interface web interactive pour lancer vos requêtes GraphQL
- Propriétés disponibles dans Spring Boot :
 - `spring.graphql.graphiql.enabled=true`
 - `spring.graphql.graphiql.path=/graphiql`



GraphQL



- Le GraphQL **n'est pas une solution magique**
 - Le back va tout de même fabriquer l'objet JSON complet
 - Mais, le front, peut choisir le flux qu'il souhaite recevoir
 - ❑ On y gagne tout de même en flux réseau et en **souplesse** front
- Le GraphQL Spring est aussi utilisable en WebReactive
 - Avec des Flux / Mono
- Le GraphQL est disponible '*simplement*' depuis la version 2.7.x de Spring Boot

Spring MVC : @RestController

- Le Spring a ses propres annotations, il est conseillé de les utiliser.
 - Donc de ne pas faire usage des annotations JAX-RS
- **@RestController** est une annotation de classe
 - Elle permet d'indiquer que la classe est un web service
 - Indispensable pour le contexte Spring.
 - Elle est un @Controller et un @ResponseBody
- Ne pas oublier de dire au Spring qu'il doit scanner votre package qui contient vos classes de web services
 - ❑ context:component-scan
- Si vous êtes en Spring < 4, vous devrez faire usage de l'annotation **@Controller**

Spring MVC : @ResponseStatus

- Cette annotation se place sur une méthode de contrôleur et permet de préciser l'HTTP status que vous souhaitez retourner quand aucun autre problème n'est survenu.

```
@DeleteMapping("/delete/{userId}")  
@ResponseStatus(value = HttpStatus.ACCEPTED)  
public UserDtoOut delete(@PathVariable("userId") Integer pUserId) throws Exception {  
    ...  
}
```

org.springframework.http.ResponseEntity

➤ Objet du Spring responsable de gérer

- ❑ Le **contenu** de la réponse : un Object transformé en JSON ou XML par vous ou le Spring
- ❑ Le **code de retour** : org.springframework.http.HttpStatus

➤ Rappels :

- ➡ https://fr.wikipedia.org/wiki/Liste_des_codes_HTTP
- ➡ Selon qui a fait la requête HTTP (code Java, Javascript, client web, ...) si le code retour n'est pas 200 alors il faudra tester le comportement. Un client peut traiter le body d'une réponse 500 alors qu'un autre n'en fera rien

JSON et org.springframework.http.ResponseEntity

- Exemple d'utilisation de ResponseEntity dans une méthode de WS

```
net.sf.json.JSONObject obj = new JSONObject();  
obj.put("login", user.getLogin());  
obj.put("nom", user.getNom());  
obj.put("prenom", user.getPrenom());  
obj.put("id", user.getId());  
resu = new ResponseEntity<String>(obj.toString(), org.springframework.http.HttpStatus.OK);
```

- On fabrique un objet JSon, ici via le framework SimpleJson
 - On ne fait pas usage de la mécanique automatique du Spring & Jackson ici
- On place dans la ResponseEntity le flux JSon et un HttpStatus
 - Cet objet sera notre retour de méthode

JSON et org.springframework.http.ResponseEntity

122

➤ Autre exemple d'utilisation de ResponseEntity

```
@PatchMapping("/deactivate/{userId}")
public ResponseEntity<UserDtoOut> deactivate(
    @PathVariable("userId") Integer pUserId) throws Exception {

    var result = this.service.disable(pUserId);
    var dtoOut = new UserDtoOut(result);
    return ResponseEntity.ok(dtoOut);
    // OU
    // return new ResponseEntity<>(dtoOut, HttpStatus.OK);
}
```

➤ C'est Jackson qui va se charger de transformer notre objet dtoOut en flux JSon ici

Spring MVC : Session et Request

- En JEE vous avez des objets standards
 - **HttpSession**
 - **HttpServletRequest**
 - **HttpResponse**
- Si vous en avez besoin, ajoutez simplement comme paramètre un élément de type **HttpSession**, **HttpServletRequest**, **HttpResponse**
 - Le Spring se chargera de vous les donner (il va les injecter)
- Mais attention : vous n'êtes plus strictement en RESTFULL et cela peut compliquer vos codes de tests unitaires.

Spring MVC : Gestion des erreurs

- Plutôt que de gérer les erreurs avec des try/catch dans vos méthodes, vous pouvez centraliser la gestion des erreurs dans
 - Tous vos contrôleurs à la fois
 - et/ou chaque contrôleur indépendamment

- Avantages :
 - le code de vos contrôleurs est très simple à lire
 - Vous définissez le ResponseEntity en fonction de vos erreurs à un seul endroit

Spring MVC : Gestion des erreurs

125

Pour un contrôleur :
annotation
`@org.springframework`
`work.web.bind.annotation.Exception`
`Handler`

```
1 package com.banque.web.controller.rest;
2
3 import javax.servlet.http.HttpSession;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23 @RestController
24 @RequestMapping("/authentifier")
25 public class AuthentifierRestController2 {
26     private static final Logger LOG = LogManager.getLogger();
27     @Autowired
28     private IAuthenticationService authenticationService;
29
30     @RequestMapping(value = "/byurl/{login}/{pwd}", method = RequestMethod.GET, produces = MediaType.APPLICATION_JSON_VALUE)
31     @ResponseBody
32     public ResponseEntity<IUtilisateurEntity> authentifierByUrl(@PathVariable String login, @PathVariable String pwd,
33         HttpSession session) throws Exception {
34         AuthentifierRestController2.LOG.info("authentifier byurl RS login={} pwd=Xxxx", login);
35         if (login == null || pwd == null || login.trim().length() == 0 || pwd.trim().length() == 0) {
36             throw new Exception("Usage is /{login}/{password}");
37         }
38
39         IUtilisateurEntity user = this.authenticationService.authentifier(login, pwd);
40         if (session != null) {
41             session.setAttribute("user", user);
42         }
43         return new ResponseEntity<IUtilisateurEntity>(user, HttpStatus.OK);
44     }
45
46     @ExceptionHandler(FonctionnelleException.class)
47     public ResponseEntity<FonctionnelleException> exceptionHandler(FonctionnelleException ex) {
48         ResponseEntity<FonctionnelleException> resu = new ResponseEntity<FonctionnelleException>(ex,
49             HttpStatus.BAD_REQUEST);
50         return resu;
51     }
52
53     @ExceptionHandler(ErreurTechniqueException.class)
54     public ResponseEntity<ErreurTechniqueException> exceptionHandler(ErreurTechniqueException ex) {
55         ResponseEntity<ErreurTechniqueException> resu = new ResponseEntity<ErreurTechniqueException>(ex,
56             HttpStatus.INTERNAL_SERVER_ERROR);
57         return resu;
58     }
59 }
```

Spring MVC : Gestion des erreurs

126

- Pour plusieurs contrôleurs : annotations
 - @org.springframework.web.bind.annotation.**ControllerAdvice**
 - Et @org.springframework.web.bind.annotation.**ExceptionHandler**

```
1 package com.banque.web.controller.rest;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.web.bind.annotation.ControllerAdvice;
6 import org.springframework.web.bind.annotation.ExceptionHandler;
7
8 import com.banque.service.ex.ErreurTechniqueException;
9 import com.banque.service.ex.FonctionnelleException;
10
11 @ControllerAdvice
12 public class ExceptionControllerAdvice {
13
14     @ExceptionHandler(FonctionnelleException.class)
15     public ResponseEntity<FonctionnelleException> exceptionHandler(FonctionnelleException ex) {
16         ResponseEntity<FonctionnelleException> resu = new ResponseEntity<FonctionnelleException>(ex,
17             HttpStatus.BAD_REQUEST);
18         return resu;
19     }
20
21     @ExceptionHandler(ErreurTechniqueException.class)
22     public ResponseEntity<ErreurTechniqueException> exceptionHandler(ErreurTechniqueException ex) {
23         ResponseEntity<ErreurTechniqueException> resu = new ResponseEntity<ErreurTechniqueException>(ex,
24             HttpStatus.INTERNAL_SERVER_ERROR);
25         return resu;
26     }
27 }
```

Spring MVC : Gestion des erreurs

- L'annotation **@ControllerAdvice** est paramétrable
 - **assignableTypes** : pour indiquer la liste des *class* visées (des classes annotées en **@Controller**)
 - **basePackages** : pour indiquer la liste des noms de packages contenant les **@Controller** visés
- Remarque : dans les exemples, on renvoie l'exception dans la **ResponseEntity** ⇔ c'est le framework Jackson qui se chargera de sa transformation en JSON.
 - Cette transformation est brutale et rarement utilisable, il faudra compléter le code en spécifiant un flux JSON pertinent pour vos applications

Spring MVC : Gestion des erreurs

- En **GraphQL**, l'annotation
 - `@org.springframework.web.bind.annotation.`
 - ❑ **ExceptionHandler**
- se transforme en
 - `@org.springframework.graphql.data.method.annotation.`
 - ❑ **GraphQLExceptionHandler**
- Le reste de la mécanique ne change pas.

Travaux pratiques

- Exo 15 - Modifions le site web afin de le compléter avec des web services REST



Spring MVC : HAL (HATEOAS)

- **Hypertext Application Language** est la concrétisation syntaxique de la norme **Hypermedia As The Engine Of Application State**
- De cette manière votre client REST n'a pas besoin de connaître les web services
 - Il peut les découvrir dynamiquement
 - Chaque élément JSON peut contenir des liens déjà paramétrés pour accéder à un autre web service rest
 - Il n'est plus nécessaire de fabriquer les liens côté client
- Le serveur peut changer ses web services sans impacter le client
 - A partir du moment où le client fait l'effort d'utiliser les liens (links/href)

Spring MVC : HAL (HATEOAS)

- Le format HAL se décompose en deux concepts principaux
 - **_links** : les liens associés à des libellés (ex: self, next, edit)
 - **_embedded** : des données complémentaires qui éviteront au client de renvoyer une demande
 - ❑ Par exemple la liste des plats dans un menu

Spring MVC : HAL (HATEOAS)

- Si vous souhaitez faire usage de cette norme vous pouvez l'intégrer dans vos projets Spring MVC REST via la dépendance à
 - `org.springframework.hateoas / spring-hateoas`
- Malheureusement, Spring ne va pas injecter automatiquement les links/embedded dans votre JSon, vous devrez le faire vous-même

Spring MVC : HAL (HATEOAS)

- Dans cette optique, vos objets JSon devront hériter de la classe `org.springframework.hateoas.RepresentationModel`
- Important :
 - les id **ne sont jamais présentés** comme propriétés dans les flux JSon en HATEOAS
 - ils doivent se retrouver sous la forme d'URL

Spring MVC : HAL (HATEOAS)

- Exemple d'un objet JSon paré pour HATEOAS
 - Il hérite **RepresentationModel<T>**

```
7 package com.banque.web.dto.out;
8
9 import java.math.BigDecimal;
10
11 import org.springframework.hateoas.RepresentationModel;
12
13 import com.banque.entity.impl.CompteEntity;
14
15 /**
16  * Le DTO pour les comptes.
17  */
18 public class CompteDtoOut extends RepresentationModel<CompteDtoOut> {
19
20     private String libelle;
21     private BigDecimal solde;
22     private BigDecimal decouvert;
23     private BigDecimal taux;
```

Spring MVC : HAL (HATEOAS)

- Dans vos contrôleurs REST vous retournerez des objets héritant **RepresentationModel**
- Pour chacun d'eux vous pourrez ajouter des attributs *links* via la méthode *add*
- Dans la méthode *add* vous ferez usage de la classe `org.springframework.hateoas.server.mvc.WebMvcLinkBuilder` et de ses méthodes statiques **methodOn**, **linkTo**

```
@GetMapping("/{userId}/{cptId}/{opId}")
public ResponseEntity<OperationDtoOut> voirOperation(@PathVariable int userId,
                                                    @PathVariable int cptId,
                                                    @PathVariable int opId) throws Exception {
    var op = this.operationService.select(userId, cptId, opId);
    var resu = new OperationDtoOut(op);
    // Vers lui meme
    resu.add(WebMvcLinkBuilder.linkTo(
        WebMvcLinkBuilder.methodOn(ListerOperationRestController.class).voirOperation(userId, cptId, opId)).withSelfRel());
    // Vers le compte
    resu.add(WebMvcLinkBuilder
        .linkTo(WebMvcLinkBuilder.methodOn(ListerCompteRestController.class).voirCompte(userId, cptId)).withRel("compte"));
    // Vers l'utilisateur
    resu.add(WebMvcLinkBuilder
        .linkTo(WebMvcLinkBuilder.methodOn(AuthentifieurRestController.class).voirUtilisateur(userId)).withRel("utilisateur"));
    return ResponseEntity.ok(resu);
}
```

Spring MVC : HAL (HATEOAS) Après : Avec HATEOAS

Avant : Sans HATEOAS

```
[
  {
    "id":13,
    "libelle":"Compte Courant",
    "solde":1460.00,
    "decouvert":100.00,
    "taux":null,
    "utilisateurId":2
  },
  {
    "id":16,
    "libelle":"Compte Remunéré",
    "solde":540.00,
    "decouvert":null,
    "taux":0.30,
    "utilisateurId":2
  }
]
```

```
[
  {
    "decouvert": 100.00,
    "libelle": "Compte Courant",
    "links": [
      {
        "rel": "self",
        "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/2/13"
      },
      {
        "rel": "utilisateur",
        "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/authentifier/2"
      }
    ],
    "solde": 14995.00
  },
  {
    "libelle": "Compte Remunéré",
    "links": [
      {
        "rel": "self",
        "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/2/16"
      },
      {
        "rel": "utilisateur",
        "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/authentifier/2"
      }
    ],
    "solde": 50005.00,
    "taux": 0.30
  }
]
```

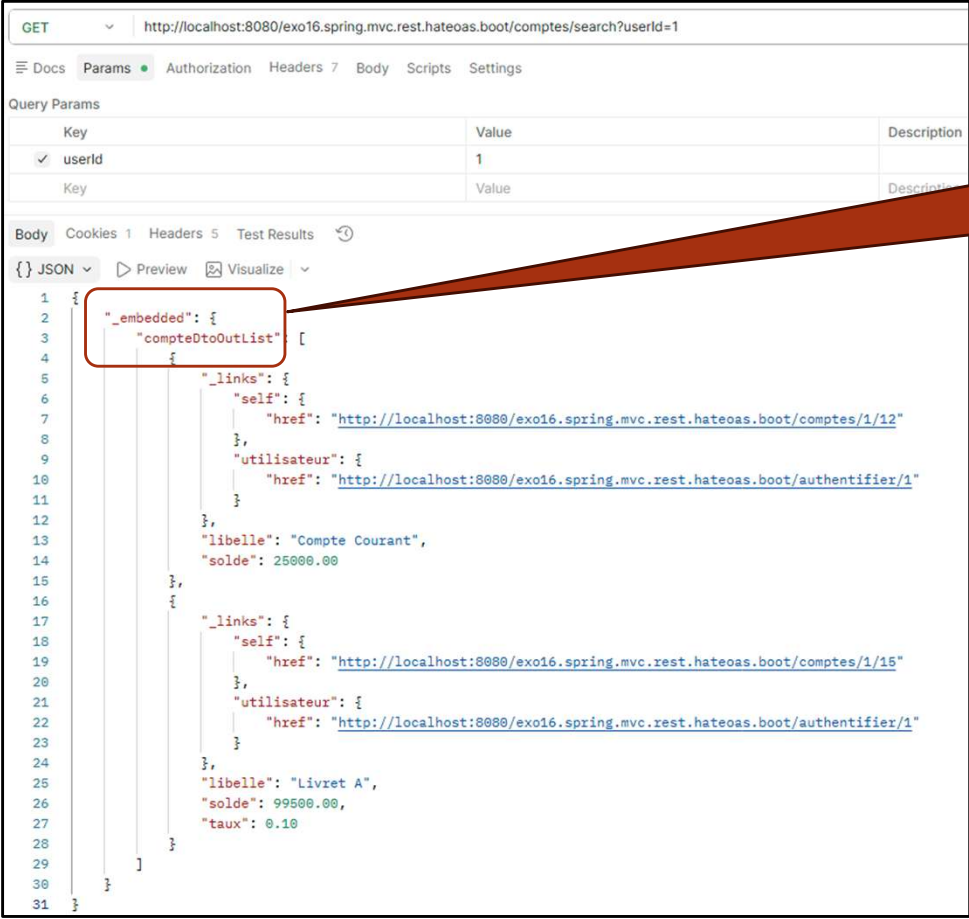

Spring MVC : HAL (HATEOAS)

- Si vous voulez faire apparaître des éléments **_embedded** il vous faudra utiliser les classes **EntityModel** et **CollectionModel** (ou **PageModel**)

```
@GetMapping("/search")
public ResponseEntity<CollectionModel<EntityModel<CompteDtoOut>>> listerCompte(@RequestParam(value = "userId") int userId)
throws Exception {
    var comptes = this.compteService.selectAll(userId);
    List<EntityModel<CompteDtoOut>> resu = new ArrayList<>(comptes.size());
    for (CompteEntity cptEntity : comptes) {
        var cj = new CompteDtoOut(cptEntity);
        // Vers lui meme
        cj.add(WebMvcLinkBuilder.linkTo(WebMvcLinkBuilder.methodOn(ListerCompteRestController.class)
            .voirCompte(cptEntity.getUtilisateur().getId(), cptEntity.getId())).withSelfRel());
        // Vers l'utilisateur
        cj.add(WebMvcLinkBuilder.linkTo(WebMvcLinkBuilder.methodOn(AuthentifierRestController.class)
            .voirUtilisateur(cptEntity.getUtilisateur().getId())).withRel("utilisateur"));
        resu.add(EntityModel.of(cj));
    }
    return ResponseEntity.ok(CollectionModel.of(resu));
}
```

Spring MVC : HAL (HATEOAS)

➤ Donnera la représentation suivante :



```
GET http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/search?userId=1

Query Params
Key      Value      Description
userId   1

Body
Cookies  1  Headers  5  Test Results

JSON
1 {
2   "_embedded": {
3     "compteDtoOutList": [
4       {
5         "_links": {
6           "self": {
7             "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/1/12"
8           },
9           "utilisateur": {
10            "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/authentifier/1"
11          }
12        },
13        "libelle": "Compte Courant",
14        "solde": 25000.00
15      },
16      {
17        "_links": {
18          "self": {
19            "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/1/15"
20          },
21          "utilisateur": {
22            "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/authentifier/1"
23          }
24        },
25        "libelle": "Livret A",
26        "solde": 99500.00,
27        "taux": 0.10
28      }
29    ]
30  }
31 }
```

Notez que le nom de la propriété n'est pas idéal (*compteDtoOutList*)

Spring MVC : HAL (HATEOAS)

➤ Si vous voulez définir vos « noms » de propriétés avec **_embedded**

➤ Ajoutez l'annotation **@Relation** sur vos DTO

```
@Relation(collectionRelation = "comptes", itemRelation = "compte")  
public class CompteDtoOut { ... }
```

Nom pour le pluriel

Nom pour le singulier

➤ Et la représentation devient

```
1 {  
2   "_embedded": {  
3     "comptes": [  
4       {  
5         "_links": {  
6           "self": {  
7             "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/1/12"  
8           },  
9           "utilisateur": {  
10            "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/authentifier/1"  
11          }  
12        },  
13        "libelle": "Compte Courant",  
14        "solde": 25000.00  
15      },  
16    ],  
17   "_links": {  
18     "self": {  
19       "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/comptes/1/15"  
20     },  
21     "utilisateur": {  
22       "href": "http://localhost:8080/exo16.spring.mvc.rest.hateoas.boot/authentifier/1"  
23     }  
24   },  
25   "libelle": "Livret A",  
26   "solde": 99500.00,  
27   "taux": 0.10  
28 },  
29 }  
30 }  
31 }
```

Spring MVC : HAL (HATEOAS)

- Dans un gros projet, la représentation d'un objet ne devra pas se copier/coller dans chaque contrôleur
- L'idée sera de réaliser, pour chaque objet modèle une classe *Assembler* qui portera la responsabilité de fabriquer l'objet `EntityModel` pour l'appliquer au flux JSON
- Cette classe implémentera l'interface **`RepresentationModelAssembler<T, EntityModel<T>>`**

Spring MVC : HAL (HATEOAS)

➤ Exemple de réalisation d'un Assembler

```
@Component // C'est @Bean que nous injecterons dans les contrôleurs qui souhaitent s'en servir
public class UserModelAssembler implements RepresentationModelAssembler<User, EntityModel<User>> {

    @Override
    public EntityModel<User> toModel(User user) {
        return EntityModel.of(user,
            linkTo(methodOn(UserController.class).getOne(user.getId())).withSelfRel(),
            linkTo(methodOn(UserController.class).getAll()).withRel("users")
        );
    }
}
```

Spring MVC : HAL (HATEOAS)

➤ Exemple d'utilisation

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserRepository repository;
    private final UserModelAssembler assembler; // Notre Assembler sera injecté

    public UserController(UserRepository repository, UserModelAssembler assembler) {
        this.repository = repository;
        this.assembler = assembler;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EntityModel<User>> getOne(@PathVariable Long id) {
        User user = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("User not found"));

        // Notre assembler portera la responsabilité de fabriquer le flux JSON
        return ResponseEntity.ok(assembler.toModel(user));
    }
}
```

Travaux pratiques

- Exo 16 - En important l'exercice HATEOAS qui reprend les web services REST
 - La dépendance est déjà dans le Maven
 - Créez les classes de beans JSon adaptés pour HATEOAS
 - Ajustez vos contrôleurs REST pour faire usage de HATEOAS



Spring et Ajax

- Rien de particulier sur ce point, le Spring ne vous aidera pas à faire de l'Ajap, vous utiliserez vos librairies Java Script habituelles :
 - JQuery
 - Angular
 - ...
- Vous ferez par contre fortement usage du JSon et donc du `@ResponseBody` et `@RequestBody`
 - Et certainement du framework Jackson